ANALISIS COMPLETO DOCFLOW8v2 — Para Reutilizacion en v3

Fecha: Abril 2026
Proposito: Documentar todo lo reutilizable del backend v2 para la version v3

---

1. ESTRUCTURA DEL PROYECTO BACKEND v2

```
C:\Desarrollo\Gentle Stack\DocFlow8v2\src\DocFlow.Backend\
├── DocFlow.sln
├── DocFlow.Api/              # Controllers (19 archivos)
├── DocFlow.Application/       # Services + DTOs + Interfaces
├── DocFlow.Domain/           # Entities
├── DocFlow.Infrastructure/    # DB Context, Repositories
├── DocFlow.Tests/            # 110+ tests
└── DocFlow.MigrationRunner/  # Migraciones
```

---

2. ENTIDADES (Domain Layer)

2.1 Entidades Core (reutilizables directamente)

Entidad	Archivo	Reutilizacion en v3
Document	`Document.cs`	✅ Clave - crear, editar, ver documentos
Expediente	`ExpedienteEntities.cs`	✅ Clave - expediente + carpetas
User	`User.cs`	✅ Clave - usuarios, roles
Organization	`Organization.cs`	⚠️ Adaptar a multi-tenant (Tenant)
DocumentType	`DocumentType.cs`	✅ Tipos documentales
Distribution	`Distribution.cs`	✅ Derivar documentos (bandeja)
DocumentTask	`DocumentTask.cs`	✅ Tareas asociadas a documentos
HistoryEntry	`HistoryEntry.cs`	✅ Historial de cambios

2.2 Detalle de Entidades

Document.cs

```
public class Document : BaseEntity
{
    public Guid OrganizationId { get; set; }        // → TenantId en v3
    public Guid DocumentTypeId { get; set; }
    public Guid CreatedByUserId { get; set; }
    public Guid CurrentHolderUserId { get; set; }
    public string Number { get; set; }             // Correlativo
```

```
    public string Subject { get; set; }            // Materia
    public string? Description { get; set; }
    public DocumentClassification Classification { get; set; }  //
Publico/Interno/Confidencial/Reservado/Secreto
    public string? Origin { get; set; }            // Remitente externo
    public DateTime? OriginDate { get; set; }
    public DateTime? ReceivedDate { get; set; }
    public DocumentStatus Status { get; set; }     // Draft/Active/Archived

    // Navigation
    public Organization Organization { get; set; } = null!;
    public DocumentType DocumentType { get; set; } = null!;
    public User CreatedByUser { get; set; } = null!;
    public User CurrentHolderUser { get; set; } = null!;
    public ICollection<Distribution> Distributions { get; set; } = new List<Distribution>();
    public ICollection<HistoryEntry> HistoryEntries { get; set; } = new List<HistoryEntry>
();
}

// Enums existentes
public enum DocumentClassification { Publico, Interno, Confidencial, Reservado, Secreto }
public enum DocumentStatus { Draft, Active, Archived }
```

ExpedienteEntities.cs

```
public class Expediente : BaseEntity
{
    public Guid OrganizationId { get; set; }
    public Guid CreatedByUserId { get; set; }
    public string Code { get; set; }               // Exp-XXXXXXXXXX
    public string Name { get; set; }               // Nombre/Materia
    public string? Description { get; set; }
    public string State { get; set; }              // Open/Closed/Archived
    public DateTime? ClosedAt { get; set; }
    public int Status { get; set; }                //
0=Creado,1=Activo,2=Cerrado,3=Archivado

    // Colecciones existentes
    public ICollection<ExpedienteDocument> Documents { get; set; } = new
List<ExpedienteDocument>();
    public ICollection<ExpedienteHito> Hitos { get; set; } = new List<ExpedienteHito>();
    public ICollection<ExpedienteShare> Shares { get; set; } = new List<ExpedienteShare>();
    public ICollection<ExpedienteChecklistItem> ChecklistItems { get; set; } = new
List<ExpedienteChecklistItem>();
    public ICollection<ExpedienteObservation> Observations { get; set; } = new
List<ExpedienteObservation>();
    public ICollection<ExpedienteHistory> History { get; set; } = new
List<ExpedienteHistory>();
}

// Entidades auxiliares (reutilizar)
```

```
public class ExpedienteDocument    // Docs asociados al expediente
public class ExpedienteHito        // Hitos/fechas importantes
public class ExpedienteShare       // Expedientes compartidos
public class ExpedienteChecklistItem  // Lista de control (checklist) ✅
public class ExpedienteObservation // Observaciones/comentarios ✅
public class ExpedienteHistory     // Historial de cambios

// Archivadores
public class Archivador { Guid OrganizationId, Guid? UserId, Name, Location, Description }
public class ArchivadorDocument { ArchivadorId, DocumentId, ArchivedByUserId }
```

User.cs

```
public class User : BaseEntity
{
    public Guid OrganizationId { get; set; }
    public Guid? OrganizationLevelId { get; set; }  // Nivel organizacional
    public string Email { get; set; }
    public string PasswordHash { get; set; }
    public string FullName { get; set; }
    public UserRole Role { get; set; } = UserRole.User  // User/Admin/SuperAdmin
    public string? Department { get; set; }
    public bool IsActive { get; set; }
    public bool MustChangePassword { get; set; }
}

public enum UserRole { User = 0, Admin = 1, SuperAdmin = 2 }
```

Organization.cs (convertir a Tenant)

```
public class Organization : BaseEntity
{
    public string Name { get; set; }           // "Municipalidad de Villarrica"
    public string Slug { get; set; }           // "villarrica"
    public string Settings { get; set; } = "{}"  // JSON config (para theming!)
    public bool IsActive { get; set; }
    public bool IsPublic { get; set; }
    public string? Rut { get; set; }
    public string? Address { get; set; }
    public string? ContactEmail { get; set; }
    public string? Phone { get; set; }
    public string? Website { get; set; }
    public int DocumentSequence { get; set; }  // Secuencial por organizacion
}
```

---

3. CONTROLLERS (API Layer)

19 Controllers existentes

Controller	Archivo	Metodos clave	Reutiliz
DocumentsController	`DocumentsController.cs`	GET/POST/PUT/DELETE documents	✅ 100
ExpedientesController	(parte de documents)	Expediente CRUD	✅ 100
UsersController	`UsersController.cs`	CRUD usuarios	✅ Adapta multi-tenant
AuthController	`AuthController.cs`	Login/Logout/Refresh	✅ JWT existent
InboxController	`InboxController.cs`	Bandeja de entrada	✅ 100
TasksController	`TasksController.cs`	Tareas	✅ 100
SearchController	`SearchController.cs`	Busqueda global	⚠️ FTS pendier
FilesController	`FilesController.cs`	Upload/Download archivos	✅ 100
OrganizationsController	`OrganizationsController.cs`	Config organizacion	⚠️ → Tenant config
ClassificationsController	`ClassificationsController.cs`	Categorias/Subcategorias	✅ 100
DocumentTypesController	`DocumentTypesController.cs`	Tipos documentales	✅ 100
ReportsController	`ReportsController.cs`	Reportes	✅ 100
ExportController	`ExportController.cs`	Exportar docs	✅ 100
TemplatesController	`TemplatesController.cs`	Plantillas	✅ 100
AccessRequestsController	`AccessRequestsController.cs`	Solicitudes acceso	✅ 100
WorkflowControllers	`WorkflowControllers.cs`	Workflows	✅ 100
PublicApiController	`PublicApiController.cs`	API publica	⚠️ Adapta multi-tenant
ExternalApiController	`ExternalApiController.cs`	API externo	⚠️ Reescril
GobIntegration/DocDigitalController		DocDigital (escaneados)	✅ 100
GobIntegration/FirmaGobController		Firma digital	✅ 100

Endpoints DocumentsController (referencia)

```
//GET /api/documents - List with pagination
//GET /api/documents/{id} - Get by ID
//POST /api/documents - Create
//PUT /api/documents/{id} - Update
//DELETE /api/documents/{id} - Delete
//GET /api/documents/{id}/history - Historial
```

---

4. SERVICIOS (Application Layer)

Interfaces existentes

Interfaz	Metodos	Reutilizar
IDocumentService	GetDocuments, CreateDocument, UpdateDocument, DeleteDocument, GetInbox, GetOutbox	✅ 100%
IDistributionService	DistributeDocument, MarkAsReceived, ReturnDocument	✅ 100% (Derivar)
IExpedienteServices	CRUD Expediente + Carpetas + Checklist	✅ 100%
IAuthService	Login, Refresh, Logout, GetCurrentUser	✅ JWT existente
IUserService	CRUD usuarios	✅ Adaptar
IOrganizationService	GetOrganization, UpdateSettings	⚠️ → Tenant config
IClassificationService	Categorias/Subcategorias	✅ 100%
ITemplateService	Plantillas	✅ 100%
IWorkflowServices	Workflows	✅ 100%
IDocDigitalService	Bandeja DocDigital	✅ 100%

DTOs existentes (importantes)

- `CreateDocumentRequest` / `UpdateDocumentRequest`
- `DocumentDto` / `DocumentListResponse`
- `LoginRequest` / `LoginResponse` / `RefreshTokenRequest`
- `DistributeDocumentRequest` / `DistributionDto`
- `ExpedienteDto` / `CreateExpedienteRequest`

---

5. AUTH (JWT) - existente

Flujo actual (v2)

```
// AuthController
[HttpPost("login")]           // Login con email/password
```

```
[HttpPost("refresh")]         // Refresh token
[HttpPost("logout")]          // Logout
[HttpGet("me")]               // Get current user

// JWT Config en appsettings
{
  "Jwt": {
    "Key": "DocFlowSecretKey...",
    "Issuer": "DocFlow",
    "Audience": "DocFlow",
    "ExpiryMinutes": 60
  }
}
```

Adaptar para v3

- Mantener JWT
- Agregar multi-tenant (claim TenantId)
- Agregar ClaveUnica (OIDC)

---

6. CARACTERISTICAS EXISTENTES (por modulo)

6.1 Documentos

- ✅ CRUD completo
- ✅ Tipos documentales configurables
- ✅ Clasificacion (Publico/Interno/Confidencial/Reservado/Secreto)
- ✅ Historial de cambios
- ✅ Upload de archivos
- ⚠️ Search con LIKE (no FTS5)

6.2 Expedientes

- ✅ Crear con codigo auto (Exp-XXXXXXXXXX)
- ✅ Carpetas dentro del expediente
- ✅ Lista de control (checklist) con %
- ✅ Compartido con usuarios
- ✅ Observaciones/comentarios
- ✅ Historial

6.3 Bandeja (Inbox/Outbox)

- ✅ Documentos recibidos
- ✅ Documentos derivados
- ✅ Marcar como recibido
- ✅ Derivar a otro usuario

6.4 Tareas

- ✅ Tareas desde documentos
- ✅ Estado Pendiente/Ejecutada
- ✅ Consulta de tareas

6.5 Integraciones

- ✅ **DocDigital**: Bandeja de documentos escaneados
- ✅ **FirmaGob**: Integracion con firma digital del gobierno
- ⚠️ **ClaveUnica**: Pendiente (no implementada)

6.6 Others

- ✅ Templates/Plantillas
- ✅ Reportes
- ✅ Export (PDF/Excel)
- ✅ Archivadores
- ✅ Clasificaciones (Categoria/Subcategoria)

---

7. TESTS EXISTENTES

```
DocFlow.Tests/
├── DocSearchServiceTests.cs        # Busqueda
├── ChileAtiendeServiceTests.cs     # Integracion ChileAtiende
├── IntegrationSyncBackgroundServiceTests.cs
├── MigrationExecutorTests.cs
├── ReportServiceTests.cs
├── ExportServiceTests.cs
├── OAuthIntegrationServiceTests.cs
├── LegacyDataTransformerTests.cs
├── DocDigitalIntegrationTests.cs   # DocDigital
├── FirmaGobIntegrationTests.cs     # Firma digital
├── ReportGenerationTests.cs
├── VariableSubstitutionEngineTests.cs
├── SignatureServiceTests.cs
├── OrganizationServiceTests.cs
├── ClassificationServiceTests.cs
├── AccessRequestServiceTests.cs
├── EmailServiceTests.cs
├── FileStorageServiceTests.cs
├── BusinessDayCalculatorTests.cs

Total: 110+ tests backend
```

---

8. BASE DE DATOS

Tablas principales (SQL Server)

```
-- Core
Documents
Expedientes
ExpedienteDocuments
ExpedienteChecklistItems
ExpedienteObservations
ExpedienteShares
Users
Organizations
```

```
DocumentTypes
Classifications (Categories/Subcategories)

-- Bandeja
Distributions

-- Tasks
DocumentTasks

-- Workflow
Workflows
WorkflowSteps

-- Files
DocumentAttachments

-- History
HistoryEntries

-- Auth
RefreshTokens
```

Migraciones

- `migration.sql` en root
- `DocFlow.MigrationRunner/` para ejecutar

---

9. LO QUE HAY QUE AGREGAR EN v3 (vs v2)

Modulo	v2 (existe)	v3 (nuevo)
Multi-tenant	❌ OrganizationId simple	✅ Tabla Tenants + DB por tenant
Theming	❌ No existe	✅ Settings JSON (colors, logo, etc)
Login + Cargo	❌ No existe UI	✅ Modal seleccion cargo
API Publica	⚠️ Basica	✅ Endpoints completos (QR, download)
ClaveUnica	❌ No implementada	✅ OIDC integration
Search	❌ LIKE	✅ FTS5
Vencimientos	⚠️ No existe en entities	✅ Agregar a Expediente
References	⚠️ No existe	✅ Sistema referencias

---

10. ESTRATEGIA DE REUTILIZACION

Copiar directamente (sin cambios)

- Entities: Document, Expediente, User, Distribution, etc.
- Services: DocumentService, ExpedienteService, DistributionService

- Controllers: DocumentsController, TasksController, InboxController
- DTOs: DocumentDto, CreateDocumentRequest, etc.

Adaptar (cambios menores)

- Organization → Tenant (agregar DB per tenant logic)
- Auth → Agregar multi-tenant claims
- Settings → Theme config (colors, logo, etc)

Reescribir (no existe)

- Login + Cargo selection UI
- Theming API
- API publica completa
- ClaveUnica OIDC
- Search FTS5
- Vencimientos en expediente

---

11. RESUMEN: % Reutilizacion

Capa	Reutilizable	Reescribir
Entities	~80%	20% (Tenant, Vencimiento)
Services	~70%	30% (Multi-tenant, Theming)
Controllers	~75%	25% (Tenant middleware, new endpoints)
Auth	~60%	40% (ClaveUnica, multi-tenant)
Frontend	0%	100% (nuevo Angular 17)

Documento de analisis para reutilizacion — Abril 2026 Proyecto: DocFlow8v3ClaudeCode